



Documentação de Teste – Sistema ReGraphik

NOVA LIMA
2026

Documentação de Teste – Sistema ReGraphik

Relatório Técnico Científico apresentado como pré-requisito para conclusão do Curso Superior de Tecnologia em Desenvolvimento de Sistemas, na CFP Escola de Programação e Robótica de Nova Lima elaborado sob orientação do Instrutor Frederico Martins Aguiar.

SUMÁRIO

1. INTRODUÇÃO.....	4
1.1. Métodos que serão testados:.....	4
1.2. Justificativa da escolha:.....	5
2. TESTE DE CHAT EM TEMPO REAL.....	5
2.1. Services.....	5
3. TESTE DE DASHBOARD.....	18
3.1. ViewModel.....	18
4. TESTE DE RELATÓRIOS ESG.....	18
4.1. ViewModel.....	18
5. PERFIL DE USUÁRIO.....	18
5.1. ViewModel.....	18

1. INTRODUÇÃO

Nome: Luna Beatriz Alve - **Equipe:** ReGraphik

Componente escolhido: Módulos de Chat em Tempo Real, Dashboard, Relatórios ESG e Perfil do Usuário

Classe ou módulo: ChatService, DashboardViewModel, RelatorioEsgViewModel e PerfilViewModel

1.1. Métodos que serão testados:

- **ChatService / ChatViewModel:** EnviarMensagemAsync, ValidarEnvio, ReceberMensagem.
- **DashboardViewModel:** CalcularIniciais, ObterFotoPerfil (propriedade FotoPerfil), FormatterTipos (formatador de eixos), CarregarIndicadores (agregação LINQ de contadores/valores), FiltrarUltimosResiduos (reindexação de IDs) e MapearCoresPizza (switch de status).
- **EsgViewModel:** ExportarPdfCommand, IrParaRelatoriosCommand, SalvarArquivo (mock do IDialogService) e ExibirConfirmacao.
- **PerfilViewModel:** AplicarMascaras, AlternarMascaraEmail (foco/desfoco), ValidarEmail, SalvarPerfilCommand, AtualizarAsync, ValidarConfirmacaoSenha e AtualizarInformacoesProfissionais.

1.2. Justificativa da escolha:

A validação destes componentes é crítica para o sistema, pois engloba:

- **Service:** Garantia de integridade do envio/recebimento de mensagens em tempo real e prevenção de falhas de comunicação.
- **ViewModel:** Validação de binding, manipulação de estados de tela (como foto, máscaras e gráficos) e resposta de comandos sem acoplamento com a interface (View).

- **Classe responsável por regras de negócio:** Lógica do chat (barrar mensagens vazias, contagem de não lidas com chat minimizado).
- **Classe de validação:** Verificação de e-mails, validação de campos obrigatórios e confirmação de senhas divergentes no perfil.
- **Classe de cálculo:** Formatação de eixos de gráficos, agregações de peso/valores monetários (LINQ) e cálculo de fallback para iniciais do avatar.
- **Componente responsável por processamento de dados:** Processamento e reindexação de resíduos recentes e exportação física de arquivos PDF.
- **Método com regras específicas:** Lógica de mascaramento/desmascaramento dinâmico de dados sensíveis (e-mail/CPF) durante o foco.

2. TESTE DE CHAT EM TEMPO REAL

2.1. Services

- **CT017 - Envio de Mensagem Válida**

Objetivo: Garantir que uma mensagem de texto simples (ex: "Resíduo #104 liberado para coleta.") seja montada com os dados corretos e pronta para ser entregue.

```

#region CT017: Testar se a mensagem é criada com todas as informações
[Fact]
0 references
public void CT017_EnviarMensagem_DeveMontarEstruturaCorreta()
{
    // Arrange: Cria uma instância de Mensagem com dados válidos
    var idMensagem = Guid.NewGuid().ToString();
    var remetente = "Usuario_A";
    var destinatario = "Usuario_B";
    var textoValido = "Resíduo #104 liberado para coleta.";

    // Act: Monta a mensagem com os dados fornecidos
    var mensagem = new Mensagem
    {
        Id = idMensagem,
        RemetenteId = remetente,
        DestinatarioId = destinatario,
        Texto = textoValido,
        DataHora = DateTime.Now,
        Lida = false
    };

    // Assert: Verifica se a mensagem foi criada corretamente
    Assert.Equal("Resíduo #104 liberado para coleta.", mensagem.Texto);
    Assert.Equal("Usuario_A", mensagem.RemetenteId);
    Assert.Equal("Usuario_B", mensagem.DestinatarioId);
    Assert.False(mensagem.Lida); // Inicia como não lida
}
#endregion

```

Resultado Obtido: O método **EnviarMensagemAsync** gerou o ID de conversa **ConversalId** e salvou o objeto no caminho **mensagens/{convId}/{mensagem.Id}**.



CT017_EnviarMensagem_Deve...

1 ms

```

/// <summary>
/// EnviarMensagemAsync
/// </summary>
/// <param name="mensagem"></param>
/// <returns></returns>
2 references | 3/3 passing
public async Task EnviarMensagemAsync(Mensagem mensagem)
{
    /// Validação: Não permitir envio de mensagens vazias
    if (mensagem == null || string.IsNullOrEmpty(mensagem.Texto))
    {
        throw new ArgumentException("Não é possível enviar mensagens vazias.");
    }

    /// Gera o ID da conversa com base nos IDs do remetente e destinatário
    var convId = ConversaId(mensagem.RemetenteId, mensagem.DestinatarioId);

    await _db
        .Child(NodeMensagens)
        .Child(convId)
        .Child(mensagem.Id)
        .PutAsync(mensagem);
}

```

- **CT018 - Bloqueio de Mensagem Vazia**

Objetivo: Garantir que o sistema impeça o envio quando o usuário tenta mandar uma mensagem sem texto (vazia, nula ou composta apenas por espaços).

```
#region CT018: Testar se o sistema recusa textos vazios ou com espaços

[Theory]
[InlineData("")] // Texto vazio
[InlineData(" ")] // Apenas espaços
[InlineData(null)] // Nulo
0 references
public async Task CT018_EnviarMensagem_TextoVazio_DeveLancarExcecao(string textoInvalido)
{
    // Arrange: Cria uma instância do serviço de chat e uma mensagem com texto inválido
    var chatService = new ChatService();
    var mensagemInvalida = new Mensagem
    {
        Id = "1",
        RemetenteId = "Usuario_A",
        DestinatarioId = "Usuario_B",
        Texto = textoInvalido
    };

    // Act & Assert: Espera-se que uma exceção seja lançada ao tentar enviar uma mensagem
    // com texto inválido
    await Assert.ThrowsAsync<ArgumentException>(() =>
        chatService.EnviarMensagemAsync(mensagemInvalida)
    );
}
#endregion
```

Resultado Obtido: O método **EnviarMensagemAsync** disparou exceção **ArgumentException** ao receber textos vazios ou espaços em branco.

>  CT018_EnviarMensagem_Texto... 1,1 sec

- Código Antigo

```
/// <summary>
/// EnviarMensagemAsync
/// </summary>
/// <param name="mensagem"></param>
/// <returns></returns>
2 references | 0/3 passing
public async Task EnviarMensagemAsync(Mensagem mensagem)
{
    var convId = ConversaId(
        mensagem.RemetenteId, mensagem.DestinatarioId);

    await _db
        .Child(NodeMensagens)
        .Child(convId)
        .Child(mensagem.Id)
        .PutAsync(mensagem);
}
```


Figura 1 - Função de enviar mensagem com erro de teste

- Código Corrigido

```
/// <summary>
/// EnviarMensagemAsync
/// </summary>
/// <param name="mensagem"></param>
/// <returns></returns>
2 references 3/3 passing
public async Task EnviarMensagemAsync(Mensagem mensagem)
{
    /// Validação: Não permitir envio de mensagens vazias
    if (mensagem == null || string.IsNullOrEmpty(mensagem.Texto))
    {
        throw new ArgumentException("Não é possível enviar mensagens vazias.");
    }

    /// Gera o ID da conversa com base nos IDs do remetente e destinatário
    var convId = ConversaId(mensagem.RemetenteId, mensagem.DestinatarioId);

    await _db
        .Child(NodeMensagens)
        .Child(convId)
        .Child(mensagem.Id)
        .PutAsync(mensagem);
}
```

- CT019 - Notificação com Chat Minimizado

Objetivo: Garantir que, quando uma nova mensagem chegar e a janela do chat não estiver focada/aberta na tela, o contador de mensagens não lidas aumente para disparar o alerta visual na barra superior.

```

#region CT019: Testar se o contador de mensagens não lidas é incrementado quando o chat está minimizado

[Fact]
0 references
public void CT019_ReceberMensagem_ComChatMinimizado_DeveIncrementarContadorNaoLidas()
{
    // Arrange: Cria uma conversa simulada e define o estado do chat como minimizado
    var conversa = new Conversa
    {
        UsuarioId = "Usuario_A",
        UsuarioNome = "Luna",
        MensagensNaoLidas = 0
    };
    bool chatEstaFocado = false;

    // Act: Simula o recebimento de uma nova mensagem enquanto o chat está minimizado
    if (!chatEstaFocado)
    {
        conversa.MensagensNaoLidas++;
    }

    // Assert: Verifica se o contador de mensagens não lidas foi incrementado corretamente
    Assert.True(conversa.MensagensNaoLidas > 0);
    Assert.Equal(1, conversa.MensagensNaoLidas);
}
#endregion

```

Resultado Obtido: Controle de estado visual corrigido.



CT019_ReceberMensagem_Co...

6 ms

```

/// <summary>
/// ContarNaoLidasAsync
/// </summary>
/// <param name="destinatarioId"></param>
/// <param name="remetenteId"></param>
/// <returns></returns>

4 references 1/1 passing
public async Task<int> ContarNaoLidasAsync(
    string destinatarioId, string remetenteId)
{
    try
    {
        var convId = ConversaId(destinatarioId, remetenteId);

        var msgs = await _db
            .Child(NodeMensagens)
            .Child(convId)
            .OnceAsync<Mensagem>();

        return msgs.Count(i =>
            i.Object.RemetenteId == remetenteId &&
            i.Object.DestinatarId == destinatarioId &&
            !i.Object.Lida);
    }
    catch { return 0; }
}

```

- CT020 — Identificador de Sala

Objetivo: Garantir que o método auxiliar ConversaId ordene os identificadores dos usuários ordinalmente para gerar a mesma chave de conversa, independentemente da ordem informada.

```

#region CT020: Testar se a geração do ID da conversa é consistente independentemente da ordem dos IDs
[Theory]
[InlineData("userA", "userB", "userA_userB")]
[InlineData("userB", "userA", "userA_userB")] // Garante ordem alfabética estática
[InlineData("123", "456", "123_456")]
0 references
public void CT020_ConversaId_OrdemIndiferente_DeveGerarMesmoIdentificador(string id1, string id2, string idEsperado)
{
    // Arrange: Obtém o método privado ConversaId via Reflection
    var metodoPrivado = typeof(ChatService)
        .GetMethod("ConversaId", BindingFlags.NonPublic | BindingFlags.Static);

    // Act: Invoca o método privado passando os dois IDs
    var resultado = metodoPrivado?.Invoke(null, new object[] { id1, id2 }) as string;

    // Assert: O ID retornado deve ser idêntico e ordenado
    Assert.Equal(idEsperado, resultado);
}

#endregion

```

Resultado Obtido: A ordenação com `Array.Sort(ids, StringComparer.Ordinal)` gerou chaves idênticas em ambos os sentidos da comunicação.

✓ CT020_Conversald_OrdemIndif...	< 1 ms
✓ CT020_Conversald_OrdemInd...	< 1 ms
✓ CT020_Conversald_OrdemInd...	< 1 ms
✓ CT020_Conversald_OrdemInd...	< 1 ms

```
/// <summary>
/// Gera um ID único para a conversa entre dois usuários, garantindo que a ordem dos IDs não importe.
/// </summary>
/// <param name="id1"></param>
/// <param name="id2"></param>
/// <returns></returns>
4 references
private static string ConversaId(string id1, string id2)
{
    var ids = new[] { id1, id2 };
    Array.Sort(ids, StringComparer.Ordinal);
    return $"{ids[0]}_{ids[1]}";
}
```

- CT021 — Marcação de Leitura no Firebase

Objetivo: Filtrar as mensagens pendentes recebidas do remetente e alterar a chave "lida" para true no registro do Firebase.

```
#region CT021: Testar a lógica de filtragem de mensagens não lidas do remetente
[Fact]
0 references
public void CT021_MarcarComoLida_LogicaFiltro_DeveFiltrarApenasMensagensNaoLidasDoRemetente()
{
    // Arrange: Prepara uma lista de mensagens simulada
    var remetenteId = "User_A";
    var destinatarioId = "User_B";

    var mensagens = new List<Mensagem>
    {
        new Mensagem { Id = "1", RemetenteId = remetenteId, DestinatarioId = destinatarioId, Lida = false }, // Deve marcar
        new Mensagem { Id = "2", RemetenteId = remetenteId, DestinatarioId = destinatarioId, Lida = true }, // Já lida
        new Mensagem { Id = "3", RemetenteId = destinatarioId, DestinatarioId = remetenteId, Lida = false } // Outro remetente
    };

    // Act: Executa o mesmo filtro Linq aplicado no MarcarComoLidaAsync
    var pendentes = mensagens
        .Where(m => m.RemetenteId == remetenteId && !m.Lida)
        .ToList();

    // Assert: Apenas a mensagem ID "1" deve ser selecionada para ser atualizada no banco
    Assert.Single(pendentes);
    Assert.Equal("1", pendentes.First().Id);
}
```

Resultado Obtido: A busca isolou as mensagens com `RemetenteId == remetenteId` e `!Lida`, atualizando individualmente a propriedade no banco.

✓ CT021_MarcarComoLida_Logi...	1 ms
--------------------------------	------

```

/// <summary>
/// MarcarComoLidaAsync
/// </summary>
/// <param name="remetenteId"></param>
/// <param name="destinatarioId"></param>
/// <returns></returns>

2 references
public async Task MarcarComoLidaAsync(
    string remetenteId, string destinatarioId)
{
    var convId = ConversaId(remetenteId, destinatarioId);

    var msgs = await _db
        .Child(NodeMensagens)
        .Child(convId)
        .OnceAsync<Mensagem>();

    var pendentes = msgs
        .Where(i => i.Object.RemetenteId == remetenteId
            && !i.Object.Lida)
        .ToList();

    foreach (var item in pendentes)
    {
        await _db
            .Child(NodeMensagens)
            .Child(convId)
            .Child(item.Key)
            .Child("lida")
            .PutAsync(true);
    }
}

```

- **CT022 — Desserialização Robusta de Usuários**

Objetivo: Mapear o nó de usuários lidando com diferenças de nomenclaturas no JSON (Nome vs name e FotoPerfil vs foto_perfil) e atribuir o ID da chave quando ausente.

```

#region CT022: Testar a lógica de parsing de JSON com nomes diferentes para o mesmo campo
[Fact]
0 references
public void CT022_ListarUsuarios_ParseJsonComNomesDiferentes_DevePreencherModelCorretamente()
{
    // Arrange: Simula retornos variados do Firebase (snake_case vs CamelCase vs name/Nome)
    string jsonBruto = @"{
        ""name"": ""Luna Beatriz"",
        ""foto_perfil"": ""https://exemplo.com/foto.png""
    }";
    string chaveFirebase = "usr_123";

    // Act: Aplica a exata lógica de tratamento presente no ListarUsuariosAsync
    var usuario = Newtonsoft.Json.JsonConvert.DeserializeObject<Usuario>(jsonBruto) ?? new Usuario();

    if (string.IsNullOrEmpty(usuario.Id))
    {
        usuario.Id = chaveFirebase;
    }

    using (var doc = JsonDocument.Parse(jsonBruto))
    {
        var root = doc.RootElement;

        if (string.IsNullOrEmpty(usuario.Nome) || usuario.Nome == "Sem Nome")
        {
            if (root.TryGetProperty("Nome", out var propName))
            {
                usuario.Nome = propName.GetString() ?? string.Empty;
            }
            else if (root.TryGetProperty("name", out var propName))
            {
                usuario.Nome = propName.GetString() ?? string.Empty;
            }
        }

        string? urlFotoDetectada = null;
        if (root.TryGetProperty("FotoPerfil", out var propFoto))
        {
            urlFotoDetectada = propFoto.GetString();
        }
        else if (root.TryGetProperty("foto_perfil", out var propFotoUnder))
        {
            urlFotoDetectada = propFotoUnder.GetString();
        }

        if (!string.IsNullOrEmpty(urlFotoDetectada))
        {
            usuario.FotoPerfil = urlFotoDetectada;
        }
    }

    // Assert
    Assert.Equal("usr_123", usuario.Id);
    Assert.Equal("Luna Beatriz", usuario.Nome);
    Assert.Equal("https://exemplo.com/foto.png", usuario.FotoPerfil);
}

#endregion

```

Resultado Obtido: Fallback de propriedades mapeou com êxito name para Nome e foto_perfil para FotoPerfil, além de capturar a chave do Firebase.



CT022_ListarUsuarios_ParseIso...

9 ms

```

/// <summary>
/// ListarUsuariosAsync - Recupera os usuários do Firebase tratando incompatibilidades de nomenclatura do JSON de forma robusta.
/// </summary>
/// <returns></returns>
4 references
public async Task<List<Usuario>> ListarUsuariosAsync()
{
    try
    {
        var items = await _db
            .Child(NodeUsuarios)
            .OnceAsync<object>();

        var listaUsuarios = new List<Usuario>();

        foreach (var item in items)
        {
            if (item.Object == null) continue;

            string jsonBruto = item.Object.ToString() ?? string.Empty;
            if (string.IsNullOrEmpty(jsonBruto)) continue;

            var usuario = Newtonsoft.Json.JsonConvert.DeserializeObject<Usuario>(jsonBruto);

            if (usuario != null)
            {
                if (string.IsNullOrEmpty(usuario.Id))
                {
                    usuario.Id = item.Key;
                }
                using (var doc = JsonDocument.Parse(jsonBruto))
                {
                    var root = doc.RootElement;

                    if (string.IsNullOrEmpty(usuario.Nome))
                    {
                        if (root.TryGetProperty("Nome", out var propName))
                        {
                            usuario.Nome = propName.GetString() ?? string.Empty;
                        }
                        else if (root.TryGetProperty("name", out var propName))
                        {
                            usuario.Nome = propName.GetString() ?? string.Empty;
                        }
                    }

                    string? urlFotoDetectada = null;
                    if (root.TryGetProperty("FotoPerfil", out var propFoto))
                    {
                        urlFotoDetectada = propFoto.GetString();
                    }
                    else if (root.TryGetProperty("foto_perfil", out var propFotoUnder))
                    {
                        urlFotoDetectada = propFotoUnder.GetString();
                    }

                    if (!string.IsNullOrEmpty(urlFotoDetectada))
                    {
                        usuario.FotoPerfil = urlFotoDetectada;
                    }
                }

                listaUsuarios.Add(usuario);
            }
        }

        return listaUsuarios;
    }
    catch (Exception ex)
    {
        System.Diagnostics.Debug.WriteLine($"[ChatService] Erro ao listar usuários: {ex.Message}");
        return [];
    }
}

```

- CT023 — Resiliência na Busca de Mensagens e Listagem

Objetivo: Tratar exceções de rede ou indisponibilidade no acesso ao Firebase sem estourar falhas na interface do usuário.

```

#region CT023: Testar comportamento de falha de rede ou Firebase ao obter mensagens
[Fact]
0 references
public async Task CT023_ObterMensagensAsync_QuandoFalharRedeOuFirebase_DeveRetornarListaVazia()
{
    // Arrange: Instancia o serviço
    var chatService = new ChatService();

    // Act: Tenta buscar mensagens em uma conversa inexistente/sem conexão
    var resultado = await chatService.ObterMensagensAsync("user1", "user2");

    // Assert: Deve acionar o bloco catch e retornar uma lista vazia sem estourar o app
    Assert.NotNull(resultado);
    Assert.Empty(resultado);
}

#endregion

```

Resultado Obtido: O bloco try/catch de **ObterMensagensAsync** e **ListarUsuariosAsync** capturou o erro e retornou uma coleção vazia.

✓ CT023_ObterMensagensAsync_... 550 ms

```

/// <summary>
/// ObterMensagensAsync
/// </summary>
/// <param name="usuarioId1"></param>
/// <param name="usuarioId2"></param>
/// <returns></returns>
3 references | 1/1 passing
public async Task<List<Mensagem>> ObterMensagensAsync(
    string usuarioId1, string usuarioId2)
{
    try
    {
        var convId = ConversaId(usuarioId1, usuarioId2);
        var items = await _db
            .Child(NodeMensagens)
            .Child(convId)
            .OnceAsync<Mensagem>();

        return items
            .Select(i => i.Object)
            .OrderBy(m => m.DataHora)
            .ToList();
    }
    catch { return []; }
}

```


- **CT024 — Resiliência na Contagem de Não Lidas**

Objetivo: Garantir retorno do valor numérico neutro 0 caso ocorra falha de comunicação com o Firebase durante a contagem de mensagens pendentes.

```
#region CT024: Testar comportamento de falha de rede ou Firebase ao contar mensagens não lidas
[Fact]
0 references
public async Task CT024_ContarNaoLidasAsync_QuandoOcorrerErro_DeveRetornarZeroSemTravar()
{
    // Arrange: Instancia o serviço
    var chatService = new ChatService();

    // Act: Executa o método de contagem de mensagens pendentes
    var totalNaoLidas = await chatService.ContarNaoLidasAsync("destinatario", "remetente");

    // Assert: Retorna 0 como valor seguro padrão no catch
    Assert.Equal(0, totalNaoLidas);
}

#endregion
```

Resultado Obtido: O método **ContarNaoLidasAsync** capturou a falha no bloco **catch** e retornou 0 como valor de fallback seguro.



CT024_ContarNaoLidasAsync_...

464 ms

```

/// <summary>
/// ContarNaoLidasAsync
/// </summary>
/// <param name="destinatarioId"></param>
/// <param name="remetenteId"></param>
/// <returns></returns>

4 references | 1/1 passing
public async Task<int> ContarNaoLidasAsync(
    string destinatarioId, string remetenteId)
{
    try
    {
        var convId = ConversaId(destinatarioId, remetenteId);

        var msgs = await _db
            .Child(NodeMensagens)
            .Child(convId)
            .OnceAsync<Mensagem>();

        return msgs.Count(i =>
            i.Object.RemetenteId == remetenteId &&
            i.Object.DestinatarioId == destinatarioId &&
            !i.Object.Lida);
    }
    catch { return 0; }
}

```

3. TESTE DE DASHBOARD

3.1. ViewModel

- CT025 — Formatação do Nome de Usuário e Perfil

Objetivo: Garantir a atribuição do nome do usuário e a normalização textual do seu perfil no construtor da ViewModel (ex: converter "Admin" para "Administrador").

```

#region CT025: Construtor e Formatação do Perfil
[Theory]
[InlineData("Admin", "Administrador")]
[InlineData("Administrador", "Administrador")]
[InlineData("Operador", "Usuário")]
[InlineData("Comum", "Usuário")]
0 references
public void CT025_Construtor_DeveFormatPerfilEAtribuirNomeCorretamente(string perfilEntrada, string perfilEsperado)
{
    // Arrange: estabelece o perfil de entrada e cria um usuário com esse perfil
    var usuario = new Usuario
    {
        Nome = "Carlos Eduardo",
        Perfil = perfilEntrada
    };

    // Act: cria a instância do DashboardViewModel com o usuário fornecido
    var viewModel = new DashboardViewModel(usuario);

    // Assert: verifica se o nome do usuário e o perfil foram atribuídos corretamente
    Assert.Equal("Carlos Eduardo", viewModel.NomeUsuario);
    Assert.Equal(perfilEsperado, viewModel.Perfil);
    Assert.NotNull(viewModel.GraficoPizzaModel);
    Assert.NotNull(viewModel.GraficoBarrasModel);
}
#endregion

```

Resultado Obtido: A ViewModel inicializou NomeUsuario e converteu "Admin" / "Administrador" para a string padronizada "Administrador", atribuindo "Usuário" para os demais perfis.

✓	CT025_Construtor_DeveFormat...	48 ms
✓	CT025_Construtor_DeveForm...	< 1 ms
✓	CT025_Construtor_DeveForm...	< 1 ms
✓	CT025_Construtor_DeveForm...	48 ms
✓	CT025_Construtor_DeveForm...	< 1 ms

```

/// <summary>
/// Construtor do ViewModel do Dashboard, responsável por inicializar os gráficos, carregar os dados
/// do usuário logado e configurar o comando de atualização de dados.
/// </summary>
/// <param name="usuarioLogado"></param>
5 references 12/12 passing
public DashboardViewModel(Usuario usuarioLogado)
{
    /// Inicialização única e fixa dos Models de Gráfico
    GraficoPizzaModel = new PlotModel { Background = OxyColors.Transparent, PlotAreaBorderThickness = new OxyThickness(0), Padding = new OxyThickness(30) };
    GraficoBarrasModel = new PlotModel { Background = OxyColors.Transparent, PlotAreaBorderThickness = new OxyThickness(0) };

    /// Extrai os dados diretamente do objeto de Usuário vindo da API/Sessão
    NomeUsuario = usuarioLogado.Nome;

    /// Tratamento visual para o perfil (ex: se vier "Admin" vira "Administrador")
    Perfil = usuarioLogado.Perfil == "Admin" || usuarioLogado.Perfil == "Administrador"
        ? "Administrador"
        : "Usuário";

    AtualizarDadosCommand = new RelayCommand(async () => await CarregarDadosDaApiAsync());

    /// Vinculando evento de sessão de forma segura
    UsuarioSessaoService.Instancia.PropertyChanged += OnUsuarioSessaoPropertyChanged;

    if (!System.ComponentModel.DesignerProperties.GetIsInDesignMode(
        new System.Windows.DependencyObject()))
    {
        _ = CarregarDadosDaApiAsync();
    }
}

```

- CT026 — Cálculo das Iniciais do Usuário Logado

Objetivo: Validar a extração das iniciais a partir da propriedade NomeUsuario para ser utilizada como avatar visual secundário, incluindo o tratamento do fallback "?".

```
#region CT026: Cálculo das Iniciais
[Theory]
[InlineData("João Silva", "JS")]
[InlineData("Maria", "M")]
[InlineData("Carlos Eduardo Silva", "CS")]
[InlineData("", "?")]
[InlineData(" ", "?")]
[InlineData(null, "?")]
0 references
public void CT026_Iniciais_DeveGerarIniciaisCorretamente(string nomeInput, string inicialEsperada)
{
    // Arrange: cria um usuário com o nome fornecido e inicializa o DashboardViewModel
    var usuario = new Usuario { Nome = "Usuario Base" };
    var viewModel = new DashboardViewModel(usuario);

    // Act: atribui o nome de teste ao NomeUsuario do viewModel
    viewModel.NomeUsuario = nomeInput;

    // Assert: verifica se as iniciais calculadas correspondem ao valor esperado
    Assert.Equal(inicialEsperada, viewModel.Iniciais);
}
#endregion
```

Resultado Obtido: A propriedade calculada **Iniciais** dividiu as palavras do nome extraíndo a primeira letra do primeiro e do último nome em maiúsculas, retornando "?" caso a propriedade estivesse nula ou vazia.

✓ CT026_Iniciais_DeveGerarInici...	< 1 ms
✓ CT026_Iniciais_DeveGerarInici...	< 1 ms
✓ CT026_Iniciais_DeveGerarInici...	< 1 ms
✓ CT026_Iniciais_DeveGerarInici...	< 1 ms
✓ CT026_Iniciais_DeveGerarInici...	< 1 ms
✓ CT026_Iniciais_DeveGerarInici...	< 1 ms
✓ CT026_Iniciais_DeveGerarInici...	< 1 ms

```

/// <summary>
/// Calcula as iniciais do usuário logado para exibir como fallback no Dashboard.
/// </summary>
2 references 6/6 passing
public string Iniciais
{
    get
    {
        if (string.IsNullOrEmpty(NomeUsuario)) return "?";
        var partes = NomeUsuario.Split(' ', StringSplitOptions.RemoveEmptyEntries);
        if (partes.Length == 1) return partes[0][0].ToString().ToUpper();
        return (partes[0][0].ToString() + partes[^1][0].ToString()).ToUpper();
    }
}

```

- **CT027 — Validação do Caminho da Foto de Perfil**

Objetivo: Verificar se o caminho da imagem de perfil cadastrado no serviço de sessão é válido (URL web ou arquivo físico local existente) antes de ser consumido no XAML.

```

#region CT027: Validação da Foto de Perfil (Arquivo Local)
[Fact]
0 references
public void CT027_FotoPerfil_QuandoCaminhoInexistente_DeveRetornarNull()
{
    // Arrange: cria um usuário e inicializa o DashboardViewModel, simulando um caminho de foto inexistente
    var usuario = new Usuario { Nome = "Teste Avatar" };
    var viewModel = new DashboardViewModel(usuario);
    string caminhoInexistente = @"C:\caminho_inexistente_foto.png";

    // Simula o serviço de sessão gravando um arquivo local inexistente
    ReGraphik.Services.UsuarioSessaoService.Instancia.FotoCaminho = caminhoInexistente;

    // Act: acessa a propriedade FotoPerfil do viewModel
    var foto = viewModel.FotoPerfil;

    // Assert: verifica se a propriedade FotoPerfil retorna null quando o caminho do arquivo não existe
    Assert.Null(foto);
}
#endregion

```

Resultado Obtido: A propriedade **FotoPerfil** validou se o arquivo existia no disco via File.Exists (para caminhos locais) ou se iniciava com "http", retornando null em caso de arquivo inexistente.



CT027_FotoPerfil_QuandoCami...

< 1 ms

```

/// <summary>
/// Obtém a imagem de perfil do usuário logado a partir do caminho armazenado no serviço de sessão,
/// lendo em memória para não trancar arquivos locais do disco.
/// </summary>
/// <summary>
/// Obtém o caminho ou URL da imagem de perfil do usuário logado.
/// </summary>
2 references 1/1 passing
public string? FotoPerfil
{
    get
    {
        string? caminho = UsuarioSessaoService.Instancia.FotoCaminho;

        if (string.IsNullOrEmpty(caminho))
            return null;

        /// Se for um arquivo local, verifica se ele realmente existe antes de mandar pro XAML
        if (!caminho.StartsWith("http", StringComparison.OrdinalIgnoreCase) && !File.Exists(caminho))
            return null;

        return caminho;
    }
}

```

● CT028 — Formatação Visual do Eixo dos Gráficos

Objetivo: Garantir a formatação padrão dos valores numéricos representados no gráfico de barras com duas casas decimais e o sufixo de unidade "kg".

```

#region CT028: Formatação de Valores do Eixo Y
[Fact]
0 references
public void CT028_FormatterTipos_DeveFormatarValorComDuasCasasESufixo()
{
    /// Arrange: cria um usuário e inicializa o DashboardViewModel, definindo um valor de entrada para formatação
    var usuario = new Usuario { Nome = "Teste Formatter" };
    var viewModel = new DashboardViewModel(usuario);
    double valorEntrada = 150.5;

    /// Act: chama o método FormatterTipos para formatar o valor de entrada
    string resultado = viewModel.FormatterTipos(valorEntrada);

    /// Assert: verifica se o resultado contém o valor esperado e termina com o sufixo "kg"
    Assert.Contains("150", resultado);
    Assert.EndsWith("kg", resultado);
}
#endregion

```

Resultado Obtido: A propriedade FormatterTipos aplicou a conversão numérica textual .ToString("N2") + " kg" corretamente.



CT028_FormatterTipos_DeveFo...

1 ms

```

/// <summary>
/// Função de formatação para os valores do eixo Y do gráfico de barras, garantindo
/// que os números sejam exibidos com duas casas decimais e a unidade "kg".
/// </summary>
1 reference 1/1 passing
public Func<double, string> FormatterTipos { get; set; } = value => value.ToString("N2") + " kg";

```

- **CT029 e CT030 — Agregação de Contadores e Lista dos 5 Últimos Resíduos**

Objetivo: Calcular o consolidado do total de resíduos, resíduos a reaproveitar, estoque e o valor financeiro estimado acumulado (multiplicando a quantidade por R\$5,50). Isolar os 5 registros de resíduos mais recentes (ordenados por data de cadastro) e reatribuir IDs numéricos sequenciais (1 a 5) para exibição resumida na interface.

```
#region CT029 & CT030: Agregação de Contadores e Lista dos 5 Últimos Resíduos
[Fact]
// 0 references
public void CT029_CT030_ProcessamentoResiduos_DeveCalcularIndicadoresEReindexarUltimosCinco()
{
    // Arrange: cria uma lista simulada de resíduos com diferentes status e datas de cadastro
    var listaResiduos = new List<Residuo>();
    for (int i = 1; i <= 10; i++)
    {
        listaResiduos.Add(new Residuo
        {
            Id = $"OLD_{i}",
            TipoResiduo = "Plástico",
            Quantidade = 10,
            Status = i % 2 == 0 ? "Reaproveitado" : "Em Estoque",
            DataCadastro = DateTime.Now.AddDays(i)
        });
    }

    // Act: realiza os cálculos de contagem, soma e reindexação dos últimos cinco resíduos
    int total = listaResiduos.Count;
    int totalReaproveitar = listaResiduos.Count(r => r.Status == "Reaproveitado");
    int totalEstoque = listaResiduos.Count(r => r.Status == "Em Estoque");
    double valorMonetario = listaResiduos.Sum(r => r.Quantidade * 5.50);

    var ultimosCinco = listaResiduos
        .OrderByDescending(r => r.DataCadastro)
        .Take(5)
        .Select((r, index) => new Residuo
        {
            Id = (index + 1).ToString(),
            TipoResiduo = r.TipoResiduo,
            Quantidade = r.Quantidade
        })
        .ToList();

    // Assert: verifica se os cálculos e a reindexação dos últimos cinco resíduos estão corretos
    Assert.Equal(10, total);
    Assert.Equal(5, totalReaproveitar);
    Assert.Equal(5, totalEstoque);
    Assert.Equal(550.00, valorMonetario);
    Assert.Equal(5, ultimosCinco.Count);
    Assert.Equal("1", ultimosCinco[0].Id);
    Assert.Equal("5", ultimosCinco[4].Id);
}
#endregion
```

Resultado CT029 Obtido: O método **CarregarDadosDaApiAsync** utilizou expressões LINQ (Count e Sum) agregando as métricas e formatando o valor financeiro como moeda (C2).



CT029_CT030_ProcessamentoR...

10 ms

```
if (todosResiduos != null && todosResiduos.Any())
{
    int totalResiduos = todosResiduos.Count();
    int totalReaproveitar = todosResiduos.Count(r => string.Equals(r.Status, "Reaproveitado", StringComparison.OrdinalIgnoreCase));
    int totalEstoque = todosResiduos.Count(r => string.Equals(r.Status, "Em Estoque", StringComparison.OrdinalIgnoreCase));

    double valorCalculado = todosResiduos.Sum(r => r.Quantidade * 5.50);
    string total = valorCalculado.ToString("C2");
}
```

Resultado CT030 Obtido: A consulta ordenou os registros em ordem decrescente, isolou os 5 primeiros via Take(5) e sobrescreveu a propriedade Id com a sequência indexada (index + 1).

```
var ultimoscinco = todosResiduos.OrderByDescending(r => r.DataCadastro).Take(5).ToList();
var ultimosComIdNumerico = ultimoscinco.Select((residuo, index) => new Residuo
{
    Id = (index + 1).ToString(),
    TipoResiduo = residuo.TipoResiduo,
    Origem = residuo.Origem,
    Quantidade = residuo.Quantidade,
    DataCadastro = residuo.DataCadastro,
    Status = residuo.Status,
    Especificacao = residuo.Especificacao,
    Projeto = residuo.Projeto,
    Condicao = residuo.Condicao,
    DimensoesCm = residuo.DimensoesCm,
    DimensoesLm = residuo.DimensoesLm,
    Observacao = residuo.Observacao,
    Anexo = residuo.Anexo
}).ToList();
```

- **CT031 — Mapeamento Dinâmico de Cores no Gráfico de Pizza**

Objetivo: Mapear e atribuir cores hexadecimais/RGB personalizadas para cada fatia do gráfico de pizza de acordo com a propriedade Status do resíduo.


```
#region CT031: Mapeamento Dinâmico de Cores no Gráfico de Pizza
[Theory]
[InlineData("Disponível", 100, 116, 139)]
[InlineData("Reservado", 50, 116, 186)]
[InlineData("Em Análise", 22, 73, 162)]
[InlineData("Coletado", 13, 42, 86)]
[InlineData("Desconhecido", 203, 213, 225)]
// 0 references
public void CT031_GraficoPizza_DeveMapearCorPorStatus(string status, byte r, byte g, byte b)
{
    // Arrange & Act: Mapeia a cor correspondente ao status usando um switch expression
    OxyPlot.OxyColor corFatia = status switch
    {
        "Disponível" => OxyPlot.OxyColor.FromRgb(100, 116, 139),
        "Reservado" => OxyPlot.OxyColor.FromRgb(50, 116, 186),
        "Em Análise" => OxyPlot.OxyColor.FromRgb(22, 73, 162),
        "Coletado" => OxyPlot.OxyColor.FromRgb(13, 42, 86),
        _ => OxyPlot.OxyColor.FromRgb(203, 213, 225)
    };

    // Assert: Verifica se a cor mapeada corresponde aos valores RGB esperados
    Assert.Equal(r, corFatia.R);
    Assert.Equal(g, corFatia.G);
    Assert.Equal(b, corFatia.B);
}
#endregion
```

Resultado Obtido: A instrução switch aplicou as cores exatas para os status "Disponível", "Reservado", "Em Análise", "Coletado", "Indisponível" e atribuiu a cor default #CBD5E1 para opções não mapeadas.

✓	CT031_GraficoPizza_DeveMape...	9 ms
✓	CT031_GraficoPizza_DeveMa...	9 ms
✓	CT031_GraficoPizza_DeveMa...	< 1 ms
✓	CT031_GraficoPizza_DeveMa...	< 1 ms
✓	CT031_GraficoPizza_DeveMa...	< 1 ms
✓	CT031_GraficoPizza_DeveMa...	< 1 ms

```
/// Lógica de agrupamento e contagem de resíduos por status, garantindo
/// que cada status tenha uma cor distinta no gráfico de pizza.
foreach (var grupo in todosResiduos.GroupBy(r => r.Status))
{
    string statusLabel = string.IsNullOrEmpty(grupo.Key) ? "Não Informado" : grupo.Key;

    OxyColor corFatia = statusLabel switch
    {
        "Disponível" => OxyColor.FromRgb(100, 116, 139),    /// #64748B
        "Reservado" => OxyColor.FromRgb(50, 116, 186),     /// #3274ba
        "Em Análise" => OxyColor.FromRgb(22, 73, 162),     /// #1649a2
        "Coletado" => OxyColor.FromRgb(13, 42, 86),        /// #0d2a56
        "Indisponível" => OxyColor.FromRgb(71, 85, 105),    /// #475569
        _ => OxyColor.FromRgb(203, 213, 225)                /// #CBD5E1
    };
    pieSeries.Slices.Add(new PieSlice(statusLabel, grupo.Count()));
    coresPaleta.Add(corFatia);
}
}
```

- **CT032 — Agrupamento de Peso por Tipo de Resíduo no Gráfico de Barras**

Objetivo: Agrupar todos os resíduos pelo seu TipoResiduo, somar o peso total acumulado (Quantidade) de cada categoria e renderizar o resultado ordenado no gráfico de barras.

```
#region CT032: Agrupamento de Peso por Tipo no Gráfico de Barras
[Fact]
0 references
public void CT032_GraficoBarras_DeveAgruparESomarPesoPorTipo()
{
    // Arrange: Cria uma lista simulada de residuos com tipos e quantidades diferentes
    var residuos = new List<Residuo>
    {
        new Residuo { TipoResiduo = "Plástico", Quantidade = 50 },
        new Residuo { TipoResiduo = "Plástico", Quantidade = 30 },
        new Residuo { TipoResiduo = "Papel", Quantidade = 100 }
    };

    // Act: Agrupa os resíduos por tipo e soma as quantidades, ordenando pelo peso total
    var agrupado = residuos
        .GroupBy(r => r.TipoResiduo)
        .Select(g => new { Tipo = g.Key, PesoTotal = g.Sum(r => r.Quantidade) })
        .OrderBy(x => x.PesoTotal)
        .ToList();

    // Assert: verifica se os resíduos foram agrupados e somados corretamente
    Assert.Equal(2, agrupado.Count);
    Assert.Equal("Plástico", agrupado[0].Tipo);
    Assert.Equal(80, agrupado[0].PesoTotal);
    Assert.Equal("Papel", agrupado[1].Tipo);
    Assert.Equal(100, agrupado[1].PesoTotal);
}
#endregion
```

Resultado Obtido: A consulta agrupou os tipos de resíduo, calculou a soma das massas, ordenou em ordem crescente de peso e popularou o categoryAxis e a série BarSeries do OxyPlot.



CT032_GraficoBarras_DeveAgru...

1 ms

```

var tiposAgrupados = todosResiduos.GroupBy(r => r.TipoResiduo)
    .Select(g => new { Tipo = g.Key, PesoTotal = g.Sum(r => r.Quantidade) })
    .OrderBy(x => x.PesoTotal)
    .ToList();

var barSeries = new BarSeries
{
    FillColor = OxyColor.FromRgb(22, 73, 162),
    TextColor = OxyColor.FromRgb(30, 41, 59),
    Font = "Segoe UI",
    FontSize = 11,
    LabelFormatString = "{0:N0} kg",
    LabelPlacement = LabelPlacement.Outside,
    BarWidth = 0.6
};

/// Adiciona os valores de peso total para cada tipo de resíduo na série de barras
foreach (var item in tiposAgrupados)
    barSeries.Items.Add(new BarItem((double)item.PesoTotal));

var categoryAxis = new CategoryAxis
{
    Position = AxisPosition.Left,
    AxisLineStyle = LineStyle.None,
    TickStyle = TickStyle.None,
    Font = "Segoe UI",
    FontSize = 12,
    TextColor = OxyColor.FromRgb(71, 85, 105),
    IsZoomEnabled = false,
    IsPanEnabled = false
};

```

4. TESTE DE RELATÓRIOS ESG

4.1. ViewModel

- **CT033 — Inicialização da ViewModel e Injeção de Dependências**

Objetivo: Garantir a correta inicialização da **EsgViewModel**, verificando a vinculação dos comandos de exportação e navegação, além do acoplamento do serviço de diálogos (IDialogService).

```
#region CT033: Construtor que inicializa corretamente os comandos
[Fact]
0 references
public void CT033_Construtor_DeveInicializarComandos()
{
    // Arrange: Cria um usuário e configura os mocks
    var usuario = new Usuario { Nome = "Empresa Teste ESG" };

    // Act: Cria a instância do ViewModel
    var viewModel = new EsgViewModel(usuario, _mockCommandRelatorios.Object, _mockDialogService.Object);

    // Assert: Verifica se os comandos foram inicializados corretamente
    Assert.NotNull(viewModel.ExportarPdfCommand);
    Assert.NotNull(viewModel.IrParaRelatoriosCommand);
}
#endregion
```

Resultado Obtido: Os comandos **ExportarPdfCommand** e **IrParaRelatoriosCommand** foram instanciados corretamente e o comando de navegação injetado foi preservado.

 CT033_Construtor_DeveInicializ... < 1 ms

```
/// <summary>
/// Construtor injetável preparado para Testes Unitários
/// </summary>
/// <param name="usuario"></param>
/// <param name="irParaRelatorios"></param>
/// <param name="dialogService"></param>
4 references 3/3 passing
public EsgViewModel(Usuario usuario, ICommand irParaRelatorios, IDialogService dialogService)
{
    _usuario = usuario;
    _dialogService = dialogService;
    ExportarPdfCommand = new RelayCommand(() => ExportarPdf());
    IrParaRelatoriosCommand = irParaRelatorios;

    /// Configuração da licença Comunitária do QuestPDF
    QuestPDF.Settings.License = LicenseType.Community;
}
```

- **CT034 — Cancelamento da Seleção de Arquivo na Exportação de PDF**

Objetivo: Validar o comportamento da aplicação quando o usuário cancela a caixa de diálogo de salvamento (SaveFileDialog).

```

#region CT034: Exportar PDF quando cancelado pelo usuário não deve gerar arquivo
[Fact]
0 references
public void CT034_ExportarPdf_QuandoCanceladoPeloUsuario_NaoDeveGerarArquivo()
{
    // Arrange: Cria um usuário e configura o mock para simular cancelamento do diálogo de salvar arquivo
    var usuario = new Usuario { Nome = "Empresa Teste" };
    _mockDialogService.Setup(d => d.SalvarArquivo(It.IsAny<string>(), It.IsAny<string>(), It.IsAny<string>()))
        .Returns((string)null!);

    var viewModel = new EsgViewModel(usuario, _mockCommandRelatorios.Object, _mockDialogService.Object);

    // Act: Tenta executar o comando de exportação para PDF
    viewModel.ExportarPdfCommand.Execute(null);

    // Assert: Verifica se o método de exibição de confirmação não foi chamado, indicando que o arquivo não foi gerado
    _mockDialogService.Verify(d => d.ExibirConfirmacao(It.IsAny<string>(), It.IsAny<string>()), Times.Never);
}
#endregion

```

Resultado Obtido: O método **ExportarPdf** interrompeu a execução logo após receber o caminho nulo/vazio, sem tentar compilar o PDF nem acionar o diálogo de confirmação ou de erro.

 CT034_ExportarPdf_QuandoCa... < 1 ms

```

1 reference
private void ExportarPdf()
{
    var nomePadrao = $"Proposta_Valor_ESG-ReGraphik_{DateTime.Now:yyyyMMdd_HH:mm}.pdf";
    var caminho = _dialogService.SalvarArquivo("Salvar Proposta de Valor ESG", "PDF (*.pdf)|*.pdf", nomePadrao);
}

```

- **CT035 — Geração com Sucesso do PDF Institucional ESG (Com e Sem Usuário)**

Objetivo: Verificar a compilação real do documento PDF em disco via engine do QuestPDF, garantindo o funcionamento do layout e do fallback textual para a empresa (_usuario?.Nome ?? "Parceira Homologada").

```

#region CT035: Exportar PDF quando caminho válido deve gerar arquivo e exibir confirmação
[Fact]
// 0 references
public void CT035_ExportarPdf_QuandoCaminhoValido_DeveGerarPdfEConfirmar()
{
    // Arrange: Cria um usuário e define um caminho temporário para o PDF
    var usuario = new Usuario { Nome = "Grafica Modelo" };
    var tempPdfPath = Path.Combine(Path.GetTempPath(), "teste_esg_output.pdf");

    _mockDialogService.Setup(d => d.SalvarArquivo(It.IsAny<string>(), It.IsAny<string>(), It.IsAny<string>()))
        .Returns(tempPdfPath);
    _mockDialogService.Setup(d => d.ExibirConfirmacao(It.IsAny<string>(), It.IsAny<string>()))
        .Returns(false);

    var viewModel = new EsgViewModel(usuario, _mockCommandRelatorios.Object, _mockDialogService.Object);

    try
    {
        // Act: Executa o comando de exportação para PDF
        viewModel.ExportarPdfCommand.Execute(null);

        // Assert: Verifica se o arquivo PDF foi criado e se a confirmação foi exibida
        Assert.True(File.Exists(tempPdfPath));
        _mockDialogService.Verify(d => d.ExibirConfirmacao(It.IsAny<string>(), It.IsAny<string>()), Times.Once);
    }
    finally
    {
        if (File.Exists(tempPdfPath)) File.Delete(tempPdfPath);
    }
}
#endregion

```

Resultado Obtido: O arquivo PDF foi gerado fisicamente no caminho indicado e o diálogo de confirmação **ExibirConfirmacao** foi acionado com o caminho correspondente.

✓ CT035_ExportarPdf_QuandoCa... 224 ms

```

var nomePadrao = $"Proposta_Valor_ESG_ReGraphik_{DateTime.Now:yyyyMMdd_HH:mm}.pdf";
var caminho = _dialogService.SalvarArquivo("Salvar Proposta de Valor ESG", "PDF (*.pdf)|*.pdf", nomePadrao);

if (string.IsNullOrEmpty(caminho)) return;

```

5. PERFIL DE USUÁRIO

5.1. ViewModel

- **CT036 — Inicialização do Perfil e Máscaras de Dados**

Objetivo: Garantir que o construtor da ContaViewModel carregue os dados do usuário e aplique o mascaramento correto às informações sensíveis (CPF e e-mail).

```

#region CT036: Inicialização do Perfil e Máscaras de Dados
[Fact]
0 references
public void CT036_Construtor_DeveCarregarDadosEMascararSensriveis()
{
    // Limpa a sessão global do ambiente de teste
    UsuarioSessaoService.Instancia.FotoCaminho = null;

    // Arrange: Cria um usuário com dados completos para teste
    var usuario = new Usuario
    {
        Nome = "Ana Souza",
        Login = "anasouza",
        Perfil = "Administrador",
        CPF = "12345678900",
        Email = "ana.souza@empresa.com",
        Cargo = "Coordenadora de Sustentabilidade",
        Departamento = "Meio Ambiente",
        Telefone = "(31) 99999-8888"
    };

    // Act: Cria a instância do ViewModel com o usuário de teste
    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object);

    // Assert: Verifica se os dados foram carregados corretamente e se as informações sensíveis foram mascaradas
    Assert.Equal("Ana Souza", viewModel.Nome);
    Assert.Equal("anasouza", viewModel.Login);
    Assert.Equal("@anasouza", viewModel.LoginExibicao);
    Assert.Equal("123.***.***-***", viewModel.CPF);
    Assert.Equal("an*****@empresa.com", viewModel.EmailResumido);
    Assert.Equal("Coordenadora de Sustentabilidade", viewModel.Cargo);
    Assert.Equal("Meio Ambiente", viewModel.Departamento);
    Assert.Equal("(31) 99999-8888", viewModel.Telefone);
    Assert.Equal("AS", viewModel.Iniciais);
    Assert.True(viewModel.SemFoto);
}
#endregion

```

Resultado Obtido: A ContaViewModel é instanciada corretamente a partir dos dados do usuário. Os campos Nome, Login, Cargo, Departamento e Telefone são atribuídos sem alterações. A propriedade LoginExibicao adiciona o prefixo @, Iniciais é gerada como "AS", e os dados sensíveis são mascarados: o CPF vira "123.***.***-***" e o e-mail vira "an*****@empresa.com". Como não há foto atribuída, SemFoto retorna true.



CT036_Construtor_DeveCarreg...

5 ms

```

2 references
private void CarregarDadosNaTela()
{
    Nome = _usuarioAtual.Nome ?? string.Empty;
    Login = _usuarioAtual.Login ?? string.Empty;
    Perfil = _usuarioAtual.Perfil ?? "Usuário";
    Cargo = _usuarioAtual.Cargo ?? "Analista Operacional";
    Departamento = _usuarioAtual.Departamento ?? "Gestão Ambiental";
    Telefone = _usuarioAtual.Telefone ?? string.Empty;

    CPF = MascaraCPF(_usuarioAtual.CPF);

    _emailReal = _usuarioAtual.Email ?? string.Empty;
    Email = MascaraEmail(_emailReal);

    OnPropertyChanged(nameof(EmailResumido));
}

```

- **CT037 — Cálculo de Iniciais com Diferentes Formatos de Nome**

Objetivo: Verificar se a propriedade Iniciais calcula corretamente o monograma visual do usuário para múltiplos cenários de formatação de nome.

```

#region CT037: Cálculo de Iniciais com Diferentes Formatos de Nome
[Theory]
[InlineData("Carlos Eduardo Silva", "CS")]
[InlineData("Beatriz", "B")]
[InlineData("", "?")]
[InlineData(" ", "?")]
[InlineData(null, "?")]
0 references
public void CT037_Iniciais_DeveGerarIniciaisCorretamente(string nomeInput, string inicialEsperada)
{
    // Arrange: cria um usuário com o nome fornecido e inicializa o ContaViewModel
    var usuario = new Usuario { Nome = "Usuario Base" };
    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object);

    // Act: atualiza o nome do usuário no ViewModel para o teste
    viewModel.Nome = nomeInput;

    // Assert: verifica se as iniciais foram geradas corretamente
    Assert.Equal(inicialEsperada, viewModel.Iniciais);
}
#endregion

```

Resultado Obtido: A propriedade Iniciais calculou corretamente "CS", "B" e o fallback "?" para entradas vazias.

✓	CT037_Iniciais_DeveGerarInici...	50 ms
✓	CT037_Iniciais_DeveGerarInici...	< 1 ms
✓	CT037_Iniciais_DeveGerarInici...	< 1 ms
✓	CT037_Iniciais_DeveGerarInici...	< 1 ms
✓	CT037_Iniciais_DeveGerarInici...	50 ms
✓	CT037_Iniciais_DeveGerarInici...	< 1 ms

```

3 references 6/6 passing
public string Iniciais
{
    get
    {
        if (string.IsNullOrEmpty(Nome)) return "?";
        var partes = Nome.Split(' ', StringSplitOptions.RemoveEmptyEntries);
        if (partes.Length == 1) return partes[0][0].ToString().ToUpper();
        return (partes[0][0].ToString() + partes[1][0].ToString()).ToUpper();
    }
}

```

- CT038 — Foco no Campo de E-mail (GotFocus / LostFocus)

Objetivo: Testar a alternância entre a exibição do e-mail real para edição e o mascaramento ao perder o foco.

```

#region CT038: Foco no Campo de E-mail (GotFocus / LostFocus)
[Fact]
0 references
public void CT038_EmailFocus_DeveExibirEmailRealAoFocarEMascararAoSair()
{
    // Arrange: Cria um usuário com e-mail válido e inicializa o ContaViewModel
    var usuario = new Usuario { Email = "desenvolvedor@teste.com" };
    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object);

    // Act - Simula ganho de foco
    viewModel.EmailGotFocusCommand.Execute(null);
    string emailEmEdicao = viewModel.Email;

    // Act - Simula perda de foco
    viewModel.EmailLostFocusCommand.Execute(null);
    string emailMascarado = viewModel.Email;

    // Assert: Verifica se o e-mail real é exibido ao focar e se o e-mail é mascarado ao perder o foco
    Assert.Equal("desenvolvedor@teste.com", emailEmEdicao);
    Assert.Equal("de*****@teste.com", emailMascarado);
    Assert.Empty(viewModel.MensagemErroEmail);
}

```

Resultado Obtido: Ao executar EmailGotFocusCommand, a propriedade Email passa a exibir o endereço de e-mail real e não mascarado (desenvolvedor@teste.com). Ao executar EmailLostFocusCommand com um e-mail válido, a propriedade volta a ser formatada com a máscara de privacidade (de*****@teste.com), mantendo a mensagem de erro vazia.



CT038_EmailFocus_DeveExibirE...

< 1 ms

```
/// <summary>
/// Método chamado quando o campo de e-mail recebe foco. Ele limpa a mensagem de erro e
/// exibe o e-mail real do usuário para edição.
/// </summary>
1 reference
public void EmailGotFocus()
{
    MensagemErroEmail = string.Empty;
    Email = _emailReal;
}

/// <summary>
/// Método chamado quando o campo de e-mail perde foco. Ele valida o e-mail informado,
/// atualiza a mensagem de erro se necessário e mascara o e-mail para exibição.
/// </summary>
1 reference
public void EmailLostFocus()
{
    if (!string.IsNullOrEmpty(Email) && !Email.Contains('@'))
    {
        MensagemErroEmail = "E-mail inválido. Verifique o endereço informado.";
    }
    else
    {
        MensagemErroEmail = string.Empty;
        _emailReal = Email ?? string.Empty;
        _usuarioAtual.Email = _emailReal;
        OnPropertyChanged(nameof(EmailResumido));
    }

    Email = MascaraEmail(_emailReal);
}
```

- CT039 — Validação de Formato de E-mail no LostFocus

Objetivo: Validar a identificação de endereços de e-mail com estrutura inválida quando o usuário encerra a edição do campo.

```
#region CT039: Validação de E-mail Inválido ao Perder Foco
[Fact]
0 references
public void CT039_EmailLostFocus_QuandoEmailInvalido_DeveExibirMensagemErro()
{
    // Arrange: Cria um usuário com e-mail válido e inicializa o ContaViewModel
    var usuario = new Usuario { Email = "email_valido@teste.com" };
    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object);

    // Act: Simula ganho de foco e perda de foco com e-mail inválido
    viewModel.EmailGotFocusCommand.Execute(null);
    viewModel.Email = "emailsemarrobainvalido.com";
    viewModel.EmailLostFocusCommand.Execute(null);

    // Assert: Verifica se a mensagem de erro é exibida corretamente para o e-mail inválido
    Assert.Equal("E-mail inválido. Verifique o endereço informado.", viewModel.MensagemErroEmail);
}
#endregion
```

Resultado Obtido: Ao alterar o valor do e-mail para um texto sem o caractere @ (ex: emailsemarrobainvalido.com) e disparar a perda de foco via `EmailLostFocusCommand`, a aplicação detecta a inconsistência e atribui a mensagem "E-mail inválido. Verifique o endereço informado." à propriedade `MensagemErroEmail`.

✓ CT039_EmailLostFocus_Quand... < 1 ms

```
if (!string.IsNullOrEmpty(Email) && !Email.Contains('@'))
{
    MensagemErroEmail = "E-mail inválido. Verifique o endereço informado.";
}
```

- **CT040 — Validação de Campos Obrigatórios ao Salvar Perfil**

Objetivo: Garantir que o perfil não seja salvo caso campos obrigatórios (Nome e Login) estejam em branco ou nulos.

```
#region CT040: Validação de Campos Obrigatórios ao Salvar Perfil
[Theory]
[InlineData("", "loginTeste", "Nome e Login são obrigatórios.")]
[InlineData("Nome Teste", "", "Nome e Login são obrigatórios.")]
0 references
public void CT040_SalvarPerfilAsync_QuandoCamposObrigatoriosVazios_DeveExibirMensagemErro(
    string nome, string login, string mensagemEsperada)
{
    // Arrange: Cria um usuário com dados iniciais e inicializa o ContaViewModel com campos obrigatórios vazios
    var usuario = new Usuario { Nome = "Original", Login = "original" };
    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object)
    {
        Nome = nome,
        Login = login
    };

    // Act: Executa o comando de salvar perfil
    viewModel.SalvarCommand.Execute(null);

    // Assert: Verifica se a mensagem de erro é exibida corretamente e se o serviço de atualização não foi chamado
    Assert.Equal(mensagemEsperada, viewModel.MensagemErroGeral);
    _mockAutorizarService.Verify(a => a.AtualizarAsync(It.IsAny<string>(), It.IsAny<Usuario>()), Times.Never);
}
#endregion
```

Resultado Obtido: Se o campo Nome ou Login estiver vazio ou nulo no momento da execução do **SalvarCommand**, a operação é cancelada via return. A propriedade `MensagemErroGeral` recebe a mensagem "Nome e Login são obrigatórios." e o serviço `_autorizarService.AtualizarAsync` não é chamado nenhuma vez.

✓ CT040_SalvarPerfilAsync_Quan... < 1 ms
✓ CT040_SalvarPerfilAsync_Qua... < 1 ms
✓ CT040_SalvarPerfilAsync_Qua... < 1 ms

```

if (string.IsNullOrEmpty(Nome) || string.IsNullOrEmpty(Login))
{
    MensagemErroGeral = "Nome e Login são obrigatórios.";
    return;
}

```

- **CT041 — Atualização de Perfil com Sucesso via Serviço**

Objetivo: Validar o fluxo de alteração de dados pessoais e persistência assíncrona com dados válidos.

```

#region CT041: Atualização do Perfil com Sucesso via Serviço
[Fact]
// 0 references
public async Task CT041_SalvarPerfilAsync_QuandoDadosValidos_DeveAtualizarComSucesso()
{
    // Arrange: Cria um usuário com dados iniciais e configura o mock do serviço de autorização para retornar sucesso na atualização
    var usuario = new Usuario { Id = "USR_123", Nome = "Antigo", Login = "antigo" };

    _mockAutorizarService
        .Setup(a => a.AtualizarAsync("USR_123", It.IsAny<Usuario>()))
        .ReturnsAsync(true);

    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object)
    {
        Nome = "Novo Nome",
        Login = "novo.Login"
    };

    // Act: Executa o comando de salvar perfil
    viewModel.SalvarCommand.Execute(null);
    await Task.Delay(100);

    // Assert: Verifica se os dados do usuário foram atualizados corretamente
    Assert.Equal("Novo Nome", usuario.Nome);
    Assert.Equal("novo.login", usuario.Login);
    Assert.Empty(viewModel.MensagemErroGeral);
    _mockAutorizarService.Verify(a => a.AtualizarAsync("USR_123", usuario), Times.Once);
}
#endregion

```

Resultado Obtido: As propriedades do modelo Usuario devem ser atualizadas, a MensagemErroGeral deve permanecer vazia e o serviço **IAutorizarService.AtualizarAsync** deve ser executado exatamente uma vez.



CT041_SalvarPerfilAsync_Quan...

126 ms

```

/// Atualiza os dados do usuário atual com os valores informados na tela
_usuarioAtual.Nome = Nome;
_usuarioAtual.Login = Login;
_usuarioAtual.Email = _emailReal;
_usuarioAtual.Telefone = Telefone;
_usuarioAtual.Cargo = Cargo;
_usuarioAtual.Departamento = Departamento;

```

- **CT042 — Divergência de Confirmação de Senha**

Objetivo: Validar a verificação de igualdade entre a nova senha e a confirmação repassadas via parâmetro do comando (MultiBinding).

```
#region CT042: Validação de Senha em SalvarPerfilAsync (MultiBinding)
[Fact]
// 0 references
public void CT042_SalvarPerfilAsync_QuandoSenhasDivergentes_DeveExibirMensagemErro()
{
    // Arrange: Cria um usuário e inicializa o ContaViewModel com senhas divergentes
    var usuario = new Usuario { Id = "USR_123", Nome = "Teste", Login = "teste" };
    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object);

    var parameter = new object[] { "senha123", "senhaDiferente" };

    // Act: Executa o comando de salvar perfil com senhas divergentes
    viewModel.SalvarCommand.Execute(parameter);

    // Assert: Verifica se a mensagem de erro é exibida corretamente e se o serviço de atualização não foi chamado
    Assert.Equal("As senhas digitadas não coincidem.", viewModel.MensagemErroGeral);
    _mockAutorizarService.Verify(a => a.AtualizarAsync(It.IsAny<string>(), It.IsAny<Usuario>()), Times.Never);
}
#endregion
```

Resultado Obtido: Ao preencher Nome e Login válidos sem alterações de foto ou senha, o objeto `_usuarioAtual` é atualizado localmente e repassado ao `_autorizarService.AtualizarAsync`. Retornando true, as propriedades de exibição (Email, LoginExibicao, EmailResumido) são notificadas e atualizadas, `MensagemErroGeral` permanece vazia e a mensagem visual de sucesso é disparada.

 CT042_SalvarPerfilAsync_Quan... 1 ms

```
if (!string.IsNullOrEmpty(novaSenha) || !string.IsNullOrEmpty(confirmacaoSenha))
{
    if (novaSenha != confirmacaoSenha)
    {
        MensagemErroGeral = "As senhas digitadas não coincidem.";
        return;
    }
}
```

- **CT043 — Atualização de Senha com Confirmação Válida**

Objetivo: Garantir a alteração da senha do usuário quando os valores de nova senha e confirmação forem idênticos.

```

#region CT043: Atualização de Senha com Sucesso
[Fact]
0 references
public async Task CT043_SalvarPerfilAsync_QuandoSenhasCoincidem_DeveAtualizarSenhaEUsuario()
{
    // Arrange: Cria um usuário e configura o mock do serviço de autorização para retornar sucesso na atualização
    var usuario = new Usuario { Id = "USR_123", Nome = "Nome Original", Login = "login.original" };

    _mockAutorizarService
        .Setup(a => a.AtualizarAsync("USR_123", It.IsAny<Usuario>()))
        .ReturnsAsync(true);

    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object);

    var parameter = new object[] { "NovaSenha123!", "NovaSenha123!" };

    // Act: Executa o comando de salvar perfil com senhas coincidentes
    viewModel.SalvarCommand.Execute(parameter);
    await Task.Delay(100);

    // Assert: Verifica se a senha do usuário foi atualizada corretamente e se o serviço de atualização foi chamado
    Assert.Equal("NovaSenha123!", usuario.Senha);
    Assert.Empty(viewModel.MensagemErroGeral);
    _mockAutorizarService.Verify(a => a.AtualizarAsync("USR_123", usuario), Times.Once);
}
#endregion

```

Resultado Obtido: Ao enviar um array de objetos contendo duas senhas distintas no parâmetro do **SalvarCommand**, a comparação identifica a divergência entre nova senha e confirmação, interrompe a execução atribuindo "As senhas digitadas não coincidem." em MensagemErroGeral, e previne a chamada da API de atualização.



CT043_SalvarPerfilAsync_Quan...

117 ms

```

if (sucesso)
{
    Email = MascararEmail(_emailReal);

    if (pbSenha != null) pbSenha.Password = string.Empty;
    if (pbConfirmacao != null) pbConfirmacao.Password = string.Empty;
}

```

- **CT044 — Atualização de Informações Profissionais**

Objetivo: Verificar a persistência das alterações nos campos de Cargo, Departamento e Telefone do usuário.

```

#region CT044: Atualização de Informações Profissionais
[Fact]
// 0 references
public async Task CT044_SalvarPerfilAsync_DeveAtualizarInformacoesProfissionais()
{
    // Arrange: Cria um usuário com informações profissionais iniciais e configura o mock do serviço de autorização para retornar sucesso na atualização
    var usuario = new Usuario
    {
        Id = "USR_123",
        Nome = "Ana Souza",
        Login = "anasouza",
        Cargo = "Analista Junior",
        Departamento = "Operações",
        Telefone = "(31) 98888-8888"
    };

    _mockAutorizarService
        .Setup(a => a.AtualizarAsync("USR_123", It.IsAny<Usuario>()))
        .ReturnsAsync(true);

    var viewModel = new ContaViewModel(usuario, _mockAutorizarService.Object)
    {
        Cargo = "Gerente de Projetos",
        Departamento = "Tecnologia",
        Telefone = "(31) 98888-7777"
    };

    // Act: Executa o comando de salvar perfil para atualizar as informações profissionais
    viewModel.SalvarCommand.Execute(null);
    await Task.Delay(100);

    // Assert: Verifica se as informações profissionais do usuário foram atualizadas corretamente e se o serviço de atualização foi chamado
    Assert.Equal("Gerente de Projetos", usuario.Cargo);
    Assert.Equal("Tecnologia", usuario.Departamento);
    Assert.Equal("(31) 98888-7777", usuario.Telefone);
    _mockAutorizarService.Verify(a => a.AtualizarAsync("USR_123", usuario), Times.Once);
}
#endregion

```

Resultado Obtido: Os novos dados profissionais devem ser gravados na instância do modelo `Usuario` e propagados com sucesso através da chamada ao serviço `IAutorizarService.AtualizarAsync`.



CT044_SalvarPerfilAsync_Deve...

105 ms

```

_usuarioAtual.Telefone = Telefone;
_usuarioAtual.Cargo = Cargo;
_usuarioAtual.Departamento = Departamento;

```